

图片 / PDF 文字解析 • 实现原理与技术路线

文档版本：v1.1.1 | 构建时间：2026-09-06 配套交付： ocr-browser.html （纯前端）、
ocr_desktop.py （Python 桌面窗口）

1. 问题定义

需要解析的"不可编辑文件上的文字"分为两类，技术路线完全不同：

类型	本质	正确做法
原生 PDF（含文本层）	文字是矢量文本，非图像	文本提取，无需 OCR，100% 准确
扫描件 / 图片 PDF / 照片	文字是像素	必须 OCR 逐页识别
普通图片（PNG/JPG...）	纯像素	OCR

关键设计原则：先做"文本层提取"，提取为空或噪声再回退到 OCR。两条交付线都内置了该回退逻辑。

2. 总体技术路线（两条主线）

2.1 纯前端路线（ocr-browser.html）

- **Tesseract.js**: Google Tesseract OCR 引擎经 Emscripten 编译为 WebAssembly，在浏览器进程内直接识别图像文字。
- **pdf.js** (Mozilla)：将 PDF 页面光栅化渲染到 `<canvas>`，从而把"扫描 PDF"变成可 OCR 的图像；对原生 PDF 也可直接调用 `getTextContent()` 提取文本层。
- **数据流向**：PDF/图片 → canvas/像素 → Tesseract.js(WASM) → 文本。整条链路在客户端完成，**文件不上传任何服务器**。
- **首屏代价**：首次加载需从公网 CDN 下载 WASM 核心（~2-4MB）+ 语言包（每语言数 MB）；**CDN 不可达时会卡住**（完整依赖与失败后果见 §8）。

2.2 Python 本地路线（ocr_desktop.py）

- **OCR 引擎**：优先 `RapidOCR`（基于 ONNX Runtime 跑 PaddleOCR 模型，~80MB、CPU 快、免 PaddlePaddle 重依赖）；若未安装则回退 `pytesseract`（~10MB，最轻）。
- **PDF 文本层提取**：`pdfplumber` 提取文本/表格；失败时用 `PyMuPDF(fitz)` 把页面渲染成图片再 OCR 回退。
- **GUI**：`tkinter`（Python 标准库自带，零额外依赖），适合老旧设备。

- **线程**：OCR 在后台线程执行，避免界面卡死。

2.3 表格识别与 Excel 导出（多 Sheet）

- **排版表格检测**：`ocr-browser.html` 对识别文本按行 / 列坐标聚类，自动切分表格区域，并以 Markdown 表格符号（`| 列 | 列 |`）呈现。
- **多表合并为单个 .xlsx**：一份文档解析出的多张表格可合并导出为一个 Excel 文件，每张表占一个 Sheet（SheetJS / `xlsx.full.min.js` 生成）。
- **「Excel 仅表格」模式**：选择该导出格式时，结果区也只展示识别到的表格，符合其选项名语义。
- **文件名规则**：下载文件名采用「源文件名 + 格式后缀」，如 `xxx.pdf` → `xxx.xlsx` / `xxx.md` / `xxx.txt`。

3. 核心处理流程

3.1 图片

图片文件 → 加载为像素 → OCR 引擎(RapidOCR / Tesseract) → 文本

3.2 原生 PDF（有文本层）

PDF → pdfplumber 逐页 `extract_text()` → 若文本充足 → 直接输出（100% 准确、极快）

3.3 扫描 PDF / 图片型 PDF

PDF → `pdf.js` / PyMuPDF 逐页渲染到 `canvas(scale≈2.5, ≈180 DPI)`
→ 每页像素送 OCR → 合并各页文本 → 输出

选择 `scale=2.5` 是因为 72 DPI (`scale=1`) 准确率明显下降，2.5 兼顾清晰度与内存。绘制前填充白色背景，避免透明区域被识别为噪点。

4. 关键技术点

1. **文本层优先（OCR 回退）**：绝大多数“真 PDF”其实带文本层，直接提取既准又快；只有扫描件才需要耗时的 OCR。该策略同时降低 CPU 占用。
2. **语言包匹配**：Tesseract 语言代码如 `chi_sim+eng`；语言选错准确率骤降。前端提供中文+英文等预设。

- 3. **Worker 复用**：Tesseract.js 每次 `createWorker` 会重载引擎，应在多页循环中复用同一 worker，结束 `terminate()`。
- 4. **离线隐私**：纯前端路线全程本地计算；Python 路线亦可完全离线（仅首次 `pip install` 需网络）。
- 5. **低配兼容**：避开 PaddleOCR/Surya 等重型模型（数百 MB、推荐 GPU），在 4GB 内存老设备上用 RapidOCR/pytesseract + Tkinter。
- 6. **OCR 文本清洗（去空格 + 换行重排）**：OCR 原始输出在 CJK / 标点 / 数字之间常带多余空格，且「每视觉行一个换行」使结果区不能撑满行宽。`cleanCJK()` 去除 CJK/标点/数字间的多余空格（保留英文词间、数字间正常空格；仅处理空格/制表/不换行空格，保留换行）；`reflowText()` 将连续视觉行合并为连贯段落，仅保留列表（- / 数字. / • 开头）与分页标记（——），并在中英文字符边界补一个空格。
- 7. **排版表格检测（启发式）**：基于 pdf.js `getTextContent()` 的坐标做行聚类（y 接近归一行）→ 行内按大间隙切列 → 相邻候选行按「列数相同 + 逐格边界接近 + 垂直距离合理」合并成表。
`compatible()` 容错：表头与数据行列数允许差 1、行距上限约 3.2× 字号、列表项 / 超长单元格排除，解决真实中文 PDF 漏检；跨页重复表头在 `collectTables` 合并。
- 8. **Tesseract.js 资源加载陷阱**：`workerPath / corePath / langPath` 必须注入 `createWorker(options)` 才会传递给 Worker 线程，仅设置全局 `Tesseract.workerPath` 无效；cdnjs 缺失 `tesseract-core.wasm.js` (404) 会导致 `createWorker` 静默挂死、点击「识别」无响应，故统一改用 jsDelivr。

5. 引擎能力对照（社区评测结论，2025–2026）

引擎	准确率	体积	中文	表格/版面	许可	备注
PaddleOCR	最高(98%+)	~500MB	100+	有	Apache2	最强但重
RapidOCR	中上	~80MB	~20	基础	Apache2	轻量首选
Tesseract5	~91%(中文)	~10MB	100+	无	Apache2	最轻最快
Surya	强	~500MB	90+	有	GPL3	版面/表格，驱动 Marker
EasyOCR	中	~1.5GB	80+	无	Apache2	2026 不推荐

6. 选型建议

- 隐私敏感 / 不想装环境 / 跨设备：纯前端 `Tesseract.js + pdf.js`。
- 个人日常 / 老设备 / 干净印刷体：`pytesseract` 或 `RapidOCR + Tkinter`。
- 中文发票 / 票据 / 表格：`RapidOCR` 起步，要求极高用 `PaddleOCR`。
- 论文 / 多栏 / 复杂版面转 **Markdown**：`Surya / MinerU / Docling`。
- 批量加可搜索层：命令行 `OCRmyPDF + Tesseract`。

7. 已知限制

- 干净印刷体 300 DPI 下约 95%–98% 字符准确率；模糊照片、手写体效果差。
- 多栏 / 表格按行返回，不自动重排为阅读顺序（需版面分析引擎处理）；但 `ocr-browser.html` 已支持排版表格检测并导出 Markdown 表格 / 多 Sheet Excel（见 § 2.3）。
- 浏览器路线首次加载需联网从 CDN 下载引擎与语言包（完整依赖与失败后果见 § 8）。

8. 前置资源与依赖清单（含加载 / 安装失败后果）

两条路线都依赖一组“前置资源”，但加载方式不同，直接决定会不会“卡在下载”：

网页版：依赖在运行时由浏览器实时从公网 CDN 拉取，CDN 故障 / 被墙 / 弱网 → 卡住或功能缺失。

Python 版：依赖在安装时一次性 `pip` / `brew` 下载到本机，运行时不联网，故无此类卡顿（只需首次安装成功）。

8.1 网页版前置资源（前端 CDN 依赖，运行时实时加载）

资源（名字）	用途	加载 / CDN 失败后果
<code>tesseract.min.js</code> （Tesseract.js 主脚本）	浏览器端 OCR 引擎入口，提供 <code>Tesseract.createWorker</code> 等 API	整个识别功能无法初始化，页面报错 / 卡死
<code>pdf.min.js</code> （pdf.js 主库）	解析 PDF 文本层、把页面渲染到 canvas	PDF 无法打开 / 解析
<code>pdf.worker.min.js</code> （pdf.js 工作线程）	PDF 解析在独立 Worker 线程执行	PDF 加载停滞、进度卡住
<code>worker.min.js</code> （Tesseract.js Worker）	OCR 计算在 Web Worker 线程执行	点击「识别」后无任何响应
<code>tesseract-core.wasm.js</code> （Tesseract 核心 WASM）	实际执行文字识别的内核	卡在「加载识别内核…」，无法识别
语言训练包 <code>chi_sim.traineddata</code> / <code>eng.traineddata</code>	中 / 英文识别模型（每个约数 MB）	卡在「下载语言包 %」，无法识别
<code>xlsx.full.min.js</code> （SheetJS）	将识别出的多张表格导出为 <code>.xlsx</code> （多 Sheet）	导出 Excel 功能失败

说明：以上资源由 `ocr-browser.html` 从 jsDelivr / cdnjs 加载。若要「首次打开即离线」，需下载到本地 `vendor/` 并改本地路径，且通过本地静态服务器（如 `python3 -m http.server`）打开——浏览器禁止从 `file://` 直接启动 Worker / 加载 `.wasm`。

8.2 Python 版前置依赖（pip / 系统包，安装时一次性下载）

包 / 资源（名字）	用途	未安装 / 安装失败后果
pdfplumber	优先提取 PDF 文本层（最高效、100% 准确）	文本层 PDF 无法解析（仅当同时未装 PyMuPDF 时）
PyMuPDF（fitz）	PDF 渲染为图片供 OCR 回退；亦能提取文本层	扫描 PDF 的 OCR 回退不可用
pytesseract	调用系统 Tesseract 做 OCR 的 Python 封装	失去一个 OCR 引擎（若也未装 RapidOCR）
pillow（PIL）	读取图片，配合 pytesseract	pytesseract 图片路径不可用
rapidocr-onnx	备用 OCR 引擎（中文更准，ONNX 模型随包捆绑）	仅失去「更准」选项，自动回退 pytesseract
系统 tesseract 二进制 + tesseract-ocr-chi-sim（语言包）	pytesseract 实际调用的 OCR 引擎与中文模型	pytesseract 引擎不可用（需 brew install tesseract 等）
tkinter（Python 标准库）	桌面窗口 GUI	仅无 GUI（标准库通常自带，无需额外装）

说明：Python 版依赖**安装后存于本机**，运行全程不联网，不存在网页版「运行时 CDN 卡住」问题。唯一代价是首次 `pip install / brew install` 需联网成功一次。

本文档与 `ocr-browser.html`、`ocr_desktop.py` 一同打包，全部为免费、开源、可离线方案。